6. テスト仕様書・結果

QCalc 1.4.0 は PC 上の模擬環境で全項目に合格しています（2026-11-13 に実施）。実機でのテストはまだ行っていません。

6.1 テストの方針と環境

実機が手元にないため、PC 上で 4 つの部分（計算エンジン、Android の Java、画面、APK と App Bundle）を分けて確かめました。計算結果の正しさは、同じソースから PC 向けに作った公式の qalc と並べて確かめています。版 1.1.0 では、解析（微分・積分・極限・微分方程式）と行列、日本の単位、キーのタブ、式の例、元に戻す、文字の大きさの試験を加え、公式 qalc との比較を 30 式から 309 式に増やしました。また、計算エンジンのソースを PC 向けにメモリの検査（AddressSanitizer と UndefinedBehaviorSanitizer）付きでビルドし、同じ試験とランダムな要求で、メモリの誤りや未定義の動作がないことを確かめました。版 1.2.0 では、精度の上限を 200000 桁に上げたことに合わせて、高精度の計算（H01〜H16）、ans の全桁の引き継ぎ（J15〜J21）、長い結果の画面（L01〜L32）の試験を加え、結果の全部の桁を別のライブラリ（Python の mpmath）と照合しました。あわせて、公式 qalc との比較 309 式、Java の 24 項目、履歴の再利用・入力・操作の画面 21 項目、APK と App Bundle の検証 22 項目を 1.2.0 でやり直しました。版 1.3.0 では、英語と日本語の画面、QCalc Pro、配色テーマ、履歴の書き出し（CSV）、プレビューの全桁表示、桁区切りの試験（Q01〜Q27）を加えました。あわせて、公式 qalc との比較 309 式、計算エンジンの機能試験・3D グラフ・解析と行列・日本の単位、式の例（日本語と英語の画面の両方）、Java の 31 項目、日本語の画面の試験（キーのタブ、式の例・元に戻す・文字の大きさ・検索、3D グラフ、見直しで直した点）、APK と App Bundle の検証 22 項目を 1.3.0 でやり直しました（2026-11-13）。高精度の計算（H）、長い結果の画面（L）、履歴の再利用・入力・操作（U）は、このときはやり直さず、版 1.2.0 の結果のままでした（版 1.3.0 で変わった点は各表の下に記しました）。版 1.4.0 では、入力の候補と変換先の候補、履歴の行、ライト／ダークの設定、横向きの画面、長い数の折り返しの試験（T01〜T27）と、公式 qalc との比較の「to mixed」8 式を加えました。あわせて、高精度の計算（H）を除くすべての試験を 1.4.0 でやり直しました（2026-11-13）。長い結果の画面（L）と履歴の再利用・入力・操作（U）も、日本語の画面でやり直しています。高精度の計算（H）は版 1.2.0 の結果です。



テストの構成 · 4 つの部分と試験の道具

項目	内容
OS	Ubuntu 24.04.5 LTS（x86-64）
計算エンジン	APK と App Bundle に入っているものと同じエンジン（版 1.4.0 は SHA-256 ac8f1bd9…b9a7、7,588,488 バイト。版 1.3.0 は 1b57c372…102e、7,588,488 バイト。版 1.2.0 は 56576cc9…1a50、7,587,080 バイト。版 1.1.0 の試験は 09ec8685…ce07a、7,582,440 バイト）を qemu-aarch64-static（8.2.2）で実行
比較用の公式 qalc	libqalculate のタグ v5.13.2（コミット d3672af9）を PC 向けにビルドした qalc。qalc -defaults（初期設定）で計算
メモリの検査	qalc-engine.cc を、PC 向けにビルドした libqalculate 5.13.2 とつないで g++ -fsanitize=address,undefined でビルドしたもの（AddressSanitizer と UndefinedBehaviorSanitizer）。検査の報告（標準エラー出力）をファイルに記録する

項目	内容
Java	OpenJDK 21.0.12 で Engine.java、StateStore.java（パッケージ io.github.hiroshitsu.qcalc）をそのまま動かす。Android の Context と SharedPreferences は試験用の小さな代用クラス、org.json は Ubuntu の com.android.json
画面	app/assets を Chromium（Playwright）で表示し、要求を上のエンジンに中継。表示サイズは 412×854（AQUOS sense8 の画面に近い大きさ。3D グラフの試験は 405×720。キーのタブの試験は 360×740、320×568、横向き 740×360 でも確かめた。版 1.3.0 の Q01〜Q27 は 412×915 で、英語と日本語の画面）
APK と App Bundle	apksigner 31.0.2、zipalign 10.0.0、aapt2 2.19、jarsigner（OpenJDK 21.0.12）、bundletool 1.17.2（ダウンロードしたものなので、ネットワークから切り離した専用のユーザーで実行）

結果のまとめ

区分	項目数	合格
公式 qalc との比較（6.2。1.4.0 でやり直し）	317	317
計算エンジンの機能試験（6.2。1.4.0 でやり直し）	26	26
3D グラフの計算（plot3d）（6.2。1.4.0 でやり直し）	10	10
解析と行列（6.2。1.4.0 でやり直し）	19	19
日本の単位（6.2。1.4.0 でやり直し）	11	11
式の例をすべて計算（6.2。1.4.0 の日本語と英語の画面の式で）	73	73
Java（本物のエンジン）（6.3。1.4.0 でやり直し）	14	14
Java（試験用の偽エンジン）（6.3。1.4.0 でやり直し）	10	10
キーのタブ・解析・行列の画面（6.4。1.4.0 でやり直し）	26	26
式の例・元に戻す・文字の大きさ・検索（6.4。1.4.0 でやり直し）	22	22
履歴の再利用・入力の変換・操作（6.4。1.4.0 でやり直し）	21	21

区分	項目数	合格
3D グラフの画面（6.4。1.4.0 でやり直し）	10	10
見直しで直した点の確認（6.4。1.4.0 でやり直し）	10	10
APK と App Bundle の検証（6.5。1.4.0 のファイルで）	22	22
高精度の計算（6.2。版 1.2.0）	16	16
Java（ans の全桁の引き継ぎ）（6.3。1.4.0 でやり直し）	7	7
長い結果の画面（6.4。版 1.2.0。1.4.0 でやり直し）	32	32
版 1.3.0 の画面（QCalc Pro、配色テーマ、書き出し、プレビュー、桁区切り）（6.4。1.4.0 でやり直し）	19	19
表示の言語（6.4。版 1.3.0。1.4.0 でやり直し）	7	7
英語の画面に日本語が残っていないこと（6.4。版 1.3.0。1.4.0 でやり直し）	1	1
版 1.4.0 の画面（入力の候補、履歴、ライト／ダーク、横向き、折り返し）（6.4）	27	27
合計	700	700

このほか、メモリの検査用のビルドで機能試験とランダムな要求 33,000 件を、配布するエンジンでランダムな要求 6,000 件を試し、異常はありませんでした（6.2 の最後）。版 1.2.0 でも、公式 qalc との比較と長い結果の画面の試験を、メモリの検査用のビルドでも実行し、誤りの報告はありませんでした。版 1.4.0 では、画面のキー、タブ、メニュー、履歴をランダムに操作する試験（360×640 の画面で、3 通りの乱数で 350 回ずつ）も行い、ページのスクリプトのエラーは 0 件でした。

6.2 計算エンジンのテスト項目と結果

公式 qalc との比較（317 件すべて一致）

同じ式を公式の qalc とアプリのエンジン（初期設定）で計算し、結果の文字列が一致することを確かめました。qalc は「式 = 結果」の形で表示するので、その結果の部分と比べています。版 1.1.0 では 309 式に、版 1.4.0 では「to mixed」の 8 式を加えて 317 式に増やしました（内訳は下の表）。下に、版 1.0.0 から続けている基本の 30 式と、版 1.1.0 で加えた解析・行列の 56 式を載せます。日本の単位の式（1 tsubo to m^2）は公式の qalc にないので比較から外

し、日本の単位の試験（N01〜N11）で確かめています。版 1.2.0 と版 1.3.0 でも同じ 309 式を比べ、すべて一致しました（版 1.3.0 のエンジンは qemu で 16 秒）。版 1.4.0 では 317 式を比べ、すべて一致しました（qemu で 17 秒）。

No	式	結果（公式 qalc とアプリのエンジンで一致）
C-01	`1+2*3`	7
C-02	`1/3 + 1/4`	7/12 ≈ 0.5833333333
C-03	`0.1 + 0.2`	0.3
C-04	`sqrt(8)`	2 × √(2) ≈ 2.828427125
C-05	`sin(30 deg)`	1/2 = 0.5
C-06	`2^100`	1.267650600E30
C-07	`100!`	9.332621544E157
C-08	`5 km to mi`	3 mi + 188 yd + 2.393700787 in
C-09	`60 km/h * 2.5 h`	150 km
C-10	`20 °C to °F`	68 °F
C-11	`solve(x^2 - 5x + 6)`	[3 2]
C-12	`x^2 = 4`	x = 2 or x = −2
C-13	`integrate(x^2 sin(x), 0, pi)`	π² − 4 ≈ 5.869604401
C-14	`diff(x^3)`	3x²
C-15	`sum(n^2, 1, 10)`	385
C-16	`[[1,2],[3,4]]^-1`	[−2 1; 3/2 −1/2] = [−2 1; 1.5 −0.5]
C-17	`255 to hex`	0xFF
C-18	`0b1010 + 0x1F`	41
C-19	`factor 360`	2³ × 3² × 5
C-20	`gcd(12, 18)`	6
C-21	`2000 * 10%`	200
C-22	`10 mod 3`	1
C-23	`(1 + 2i)^2`	−3 + 4i
C-24	`e^(i*pi)`	−1

No	式	結果（公式 qalc とアプリのエンジンで一致）
C-25	`(5 +/- 0.2) m * (3 +/- 0.1) m`	15.00±0.78 m²
C-26	`1 GiB to MB`	1073.741824 MB
C-27	`speed_of_light to km/s`	299792.458 km/s
C-28	`"2026-11-09" + 100 days`	"2027-02-17"
C-29	`7/3`	2 + 1/3 ≈ 2.333333333
C-30	`cos(pi/3) + ln(e^2)`	5/2 = 2 + 1/2 = 2.5

比較した式の内訳

式の組	式の数	主な内容
C-01〜C-30（基本の 30 式）	30	四則、根、三角関数、単位、方程式、積分、微分、総和、逆行列、進数、因数分解、パーセント、剰余、複素数、誤差、物理定数、日付
C-31〜C-253（広い範囲）	223	関数（階乗・組合せ・丸め・指数・対数・三角・双曲線）、複素数、単位の変換、方程式・不等式・連立方程式、微分・積分・極限・総和、行列とベクトルの基本、素数と因数分解、ビット演算、2・8・16 進数とローマ数字、分数、日付と時間、誤差、物理定数、統計、特殊関数、文字列、記号の式（因数分解・展開）、厳密な形を近似値の前に出すかどうかの境目（長い厳密な形）
C-254〜C-309（解析と行列）	56	微分 7、極値 1、積分 10、数値積分 1、極限 1、総和・積 3、微分方程式 3、数値解・臨界点 2、特殊関数 3、複素数 1、行列・ベクトル 24
C-310〜C-317（単位を混ぜた表し方。版 1.4.0）	8	「to mixed」（長さ、質量、時間）。追加した日本の単位が結果に出ないことの確認

解析と行列の比較（版 1.1.0 で加えた 56 式）

No	式	結果（公式 qalc とアプリのエンジンで一致）
C-254	diff(x^3 + 2x)	3x² + 2
C-255	diff(x^2 × sin(x))	cos(x) × x² + 2 × sin(x) × x
C-256	diff(x^5, x, 2)	20x³
C-257	diff(x^2 × y^3, y)	3x²y²
C-258	diff(x^3, x, 1, 2)	12
C-259	diff(ln(x^2 + 1))	(2x) / (x² + 1)
C-260	diff(sin(x)^2, x, 3)	−4 × sin(2x)
C-261	extremum(x^3 − 3x)	[1 −1]
C-262	integrate(1/x)	ln(|x|) + C
C-263	integrate(x^2 × sin(x), 0, π)	π² − 4 ≈ 5.869604401
C-264	integrate(√(1 − x^2), −1, 1)	π / 2 ≈ 1.570796327
C-265	integrate(e^(−x^2), −∞, ∞)	√(π) ≈ 1.772453851
C-266	integrate(1/x^2, 1, ∞)	1
C-267	integrate(1/(1 + x^2), 0, ∞)	π / 2 ≈ 1.570796327
C-268	integrate(integrate(x × y, 0, 1, x), 0, 2, y)	1
C-269	integrate(e^(−x^2), 0, 1)	(√(π) × erf(1)) / 2 ≈ 0.7468241328

No	式	結果（公式 qalc とアプリのエンジンで一致）
C-270	integrate(x × ln(x), 1, e)	(e² + 1) / 4 ≈ 2.097264025
C-271	integrate(tan(x))	−ln(|cos(x)|) + C
C-272	romberg(sin(x)/x, 1, 2)	0.6593299064
C-273	limit((x^2 − 1)/(x − 1), 1)	2
C-274	sum(x^2, 1, 10)	385
C-275	sum(1/2^n, 0, 20, n)	1.999999046
C-276	product(x, 1, 5)	120
C-277	dsolve(diff(y, x) = y)	e^x × C ≈ 2.718281828^x × C
C-278	dsolve(diff(y, x) = −2y, 5)	5 / e^(2x) ≈ 5.000000000 × 0.1353352832^x
C-279	dsolve(diff(y, x) + 2y = e^x, 1)	e^x / 3 + 2 / (3e^(2x)) ≈ 0.3333333333 × 2.718281828^x + 0.6666666667 × 0.1353352832^x
C-280	newtonsolve(cos(x) = x, 1)	0.7390851332
C-281	solve(diff(x^3 − 3x) = 0)	[1 −1]
C-282	gamma(1/2)	√(π) ≈ 1.772453851
C-283	zeta(2)	π² / 6 ≈ 1.644934067
C-284	Si(1)	0.9460830704
C-285	[1, 2; 3, 4]	[1 2; 3 4]
C-286	[1 2; 3 4]	[1 2; 3 4]

No	式	結果（公式 qalc とアプリのエンジンで一致）
C-287	[1, 2; 3, 4] × [5, 6; 7, 8]	[19 22; 43 50]
C-288	[1, 2; 3, 4]^−1	[−2 1; 3/2 −1/2] = [−2 1; 1.5 −0.5]
C-289	[1, 2; 3, 4]^2	[7 10; 15 22]
C-290	2 × [1, 2; 3, 4] + 1	[3 5; 7 9]
C-291	det([1, 2, 3; 4, 5, 6; 7, 8, 10])	−3
C-292	transpose([1, 2, 3; 4, 5, 6])	[1 4; 2 5; 3 6]
C-293	adj([1, 2; 3, 4])	[4 −2; −3 1]
C-294	rk([1, 2; 2, 4])	1
C-295	rref([2, 1, 3; 1, 3, 5])	[1 0 4/5; 0 1 7/5] = [1 0 0.8; 0 1 1.4]
C-296	identity(3)	[1 0 0; 0 1 0; 0 0 1]
C-297	[2, 1; 1, 3]^−1 × [3; 5]	[4/5; 7/5] = [0.8; 1.4]
C-298	multisolve([2x + y = 3, x + 3y = 5], [x, y])	[4/5 7/5] = [0.8 1.4]
C-299	solve(det([2, 1; 1, 2] − x × identity(2)) = 0)	[3 1]
C-300	[1, 2; 3, 4] × [x; y]	[(x + 2y); (3x + 4y)]
C-301	[[1, 2, 3], [4, 5, 6], [7, 8, 10]]^−1	[−2/3 −4/3 1; −2/3 11/3 −2; 1 −2 1] ≈ [−0.6666666667 −1.333333333 1; −0.6666666667 3.666666667 −2; 1 −2 1]
C-302	dot([1, 2, 3], [4, 5, 6])	32

No	式	結果（公式 qalc とアプリのエンジンで一致）
C-303	norm([3, 4])	5
C-304	[1, 2, 3] + 2 × [4, 5, 6]	[9 12 15]
C-305	[1, 2, 3] × [4, 5, 6]	[1 2 3] × [4 5 6]
C-306	[1, 2; 2, 4]^−1	1 / [1 2; 2 4]
C-307	(3 + 4i) × (1 − 2i)	11 − 2i
C-308	kron([1, 2; 3, 4], [1, 0; 0, 1])	[1 0 2 0; 0 1 0 2; 3 0 4 0; 0 3 0 4]
C-309	exp([1, 2; 3, 4])	[e e^2; e^3 e^4] ≈ [2.718281828 7.389056099; 20.08553692 54.59815003]

C-258 の diff(x^3, x, 1, 2) は、x = 2 での 1 階の微分係数です。C-299 は行列 [2, 1; 1, 2] の固有値（3 と 1）を特性方程式で求めています。C-305 と C-306 は計算できない式で、エラー（ベクトルどうしの積には cross()・dot()・hadamard() を使う、逆行列がない）とともに式をそのまま返します（qalc と同じ）。

単位を混ぜた表し方の比較（版 1.4.0 で加えた 8 式）

「to mixed」は、結果を大きい単位と小さい単位を混ぜて表します。版 1.3.0 までは、追加した日本の単位が混ぜる相手に選ばれ、C-310 が「16.5 寸 + 1 m」、C-311 が「1 貫 + 1.25 kg」になって、公式の qalc と違っていました（詳細設計書 3.19）。版 1.4.0 で直し、8 式とも一致します。

No	式	結果（公式 qalc とアプリのエンジンで一致）
C-310	1.5 m to mixed	1.5 m
C-311	5 kg to mixed	10 pfund
C-312	3.7 h to mixed	3 h + 42 min
C-313	10000 s to mixed	2 h + 46 min + 40 s
C-314	100 in to mixed	8 ft + 4 in
C-315	5.5 ft to mixed	5 ft + 6 in

No	式	結果（公式 qalc とアプリのエンジンで一致）
C-316	2.5 lb to mixed	2 lb + 8 oz
C-317	1.25 mi to mixed	1 mi + 440 yd

機能試験（26 件すべて合格。版 1.4.0 のエンジンでやり直し）

エンジンを 1 つ起動し、順に要求を送って応答を確かめました（QALC_CURRENCY=JPY）。

ID	内容	確かめたこと（実際の値）	結果
E01	起動の応答	engine が libqalculate 5.13.2、新しいフォルダでは ans_defined=false	合格
E03	プレビューで代入を実行しない	x = 5 が deferred=true、続く x + 1 は x + 1 のまま	合格
E04	プレビューで save() を実行しない	deferred=true	合格
E05a	確定のあとの ans イベント	{"event":"ans","id":5,"value":"7/3","input":"7/3"}	合格
E05b	ans が表示した値と一致	ans × 3 = 7	合格
E06	方程式の解を ans に入れる	x = 2 or x = −2 の input が [2 −2]	合格
E07	ans2 に前の結果	ans2 = 7/3	合格
E08	set_ans で ans を戻す	set=true、ans + 1 = 11	合格
E09	長い結果を max_chars で切る	100!（全桁）を 100 バイトで切り「…」を付ける、truncated=true	合格
E10	壊れた JSON を断る	{"id":null,"ok":false,"error":"bad request"}、後ろに余分な文字がある場合も同じ	合格
E11	入れ子が深すぎる式を断る	301 段のかっこで「The expression is nested too deeply」（エンジンの文は版 1.3.0 から英語。日本語の画面では「式の入れ子が深すぎます」）	合格
E12	未知のコマンド	unknown command	合格

ID	内容	確かめたこと（実際の値）	結果
E13a	変数の定義と利用	r = 5 cm を保存、r × 2 = 100 mm	合格
E13b	関数の定義と利用	hyp = sqrt(x^2 + y^2) を保存、hyp(3, 4) = 5	合格
E13c	定義できないものを断る	pi・sin は組み込みの名前、1abc は使えない文字、5 +* 3 は「Check the value …」（版 1.3.0 から英語）	合格
E14	定義をファイルに保存	definitions/variables.xml、functions.xml ができ、user_defs に r と hyp	合格
E15	定義の削除	1 回目は saved=true、2 回目は「There is no user definition "r"」（版 1.3.0 から英語）	合格
E16	確定した x = 5 は一時的な変数	user_defs で temporary=true	合格
E17	グラフの点	sin(x) と 1/x の 2 系列、各 9 点（steps 10）	合格
E18	グラフの範囲と式の数の確認	min > max は invalid range、13 個は too many expressions	合格
E19	一覧	関数 426、変数 192、単位 438 件（関数は libqalculate の 424、trace()、E13b で定義した hyp。単位は日本の単位 10 を含む）	合格
E20	為替レートの取得元	eurofxref-daily.xml、btc.json、rates.json（どれも利用者フォルダの中）	合格
E21	端末の通貨	100 USD が ¥ で表示される	合格
E22	時間切れと再起動	厳密モードの fibonacci(10^9)（timeout 1500）で 4.6 秒後に timeout=true、restart=true、終了コード 3	合格
E23	長い厳密な形を近似値の前に出さない（qalc と同じ）	pi + e + sqrt(2) + ln(2) + sin(1) + cos(1) + tan(1) は 10.90641624 だけ（approx=true）。自動モードの fibonacci(10^9) は時間切れにならず、近似の形（9.694687234E… で始まる）で答える	合格

ID	内容	確かめたこと（実際の値）	結果
E24	単位の一覧で、複合単位に入力できる文字を返す	km・km/h・GB を返し、それを使った 5 km + 3 cm to m = 5000.03 m、90 km/h to m/s = 25 m/s。式に挿入できない単位は 11 個（insert=false）	合格

E21 の値はエンジンに組み込まれた為替レート（試験環境では取得できないため）によるものです。

3D グラフの計算（plot3d）（10 件すべて合格。版 1.4.0 でやり直し）

plot3d の要求をエンジンに送り、返ってくる格子と z の値を確かめました。

ID	内容	確かめたこと（実際の値）	結果
P01	格子の形と値	sin(x) cos(y)、5×5：x は −5、−2.5、0、2.5、5。z は 25 個で、最初の値が sin(−5)·cos(−5) = 0.2720105554	合格
P02	2 つの式と、実数にならない点	x*y と sqrt(x)（−1〜1、3×3）：z が [1, 0, −1, 0, 0, 0, −1, 0, 1] と [null, 0, 1, null, 0, 1, null, 0, 1]	合格
P03	極（値がない点）	1/(x*y)（5×5）：x = 0 か y = 0 の 9 点（0 から数えて 2、7、10〜14、17、22 番目）が null	合格
P04	単位の付いた値は描かない	x * 1 m：9 点すべて null	合格
P05	範囲に式を使える	x は pi〜2pi（3.14159265359、4.71238898038、6.28318530718）、y は 0〜1	合格
P06	範囲の誤りを断る	x が 5〜1 は invalid x range、y が abc は invalid y range	合格
P07	式の数の確認	5 個は too many expressions、0 個は exprs missing	合格
P08	格子の数の範囲	nx 500 は 121 に、ny 1 は 2 に直す	合格
P09	細かい格子の時間	61×61（3,721 点）を qemu で 0.2 秒	合格

ID	内容	確かめたこと（実際の値）	結果
P10	時間切れ	fibonacci(200000 + round(abs(x)))（41×41、timeout 1500）：1.5 秒で complete=false を返し、エンジンは起動し直さない	合格

解析と行列（19 件すべて合格。版 1.4.0 でやり直し）

版 1.1.0 で加えたキー（解析・行列）が入れる式と、橋渡しプログラムが加えた trace()、メッセージの保留の問題への対処を確かめました。式は、画面のキーが入れる文字（×、−、π、∞など）で書いています。

ID	内容	確かめたこと（実際の値）	結果
M01	微分（1 階・2 階・偏微分・点での値）と極値	diff(x^3 + 2x) = 3x² + 2、diff(x^5, x, 2) = 20x³、diff(x^2 × y^3, y) = 3x²y²、diff(x^3, x, 1, 2) = 12、diff(x^2 × sin(x)) = cos(x) × x² + 2 × sin(x) × x、extremum(x^3 − 3x) = [1 −1]	合格
M02	積分（不定積分・定積分・二重積分）	integrate(1/x) = ln(|x|) + C、integrate(x^2 × sin(x), 0, π) = π² − 4 ≈ 5.869604401、integrate(√(1 − x^2), −1, 1) = π / 2、integrate(integrate(x × y, 0, 1, x), 0, 2, y) = 1、内側だけなら y / 2	合格
M03	無限大を端にもつ積分	integrate(e^(−x^2), −∞, ∞) = √(π) ≈ 1.772453851、integrate(1/x^2, 1, ∞) = 1、integrate(1/(1 + x^2), 0, ∞) = π / 2、integrate(e^(−x), 0, ∞) = 1	合格
M04	M03 のあとも警告が出る（libqalculate 5.13.2 の問題への対処）	1/0 に「Division by zero.」、limit(sin(x)/x, 0) = 1 と「limit() is incomplete and unreliable.」、積分と 1/0 を含む式（= 1 + 1 / 0）にも警告、続く 1/0 にも警告	合格
M05	同じ積分を 25 回（確定とプレビューを交互に）計算したあと	integrate(e^(−x), 0, ∞) + 5 = 6 でメッセージなし。確定とプレビューの 1/0 にどちらも警告	合格

ID	内容	確かめたこと（実際の値）	結果
M06	極限（点・無限大・右から・左から）	limit(sin(x)/x, 0) = 1、limit((1 + 1/x)^x, ∞) = e、limit(1/x, 0, x, 1) = +∞、limit(1/x, 0, x, −1) = −∞、limit((x^2 − 1)/(x − 1), 1) = 2	合格
M07	総和と積	sum(x^2, 1, 10) = 385、product(x, 1, 5) = 120、sum(1/2^n, 0, 20, n) = 1.999999046、sum(x, 1, 100) = 5050	合格
M08	微分方程式・数値解・臨界点	dsolve(diff(y, x) = y) = e^x × C、dsolve(diff(y, x) = −2y, 5) = 5 / e^(2x)、dsolve(diff(y, x) = x^2) = x³ / 3 + C、newtonsolve(cos(x) = x, 1) = 0.7390851332、solve(diff(x^3 − 3x) = 0) = [1 −1]	合格
M09	特殊関数	gamma(1/2) = √(π)、zeta(2) = π² / 6、erf(1) = 0.8427007929、Si(1) = 0.9460830704、integrate(e^(−x^2), 0, 1) = (√(π) × erf(1)) / 2	合格
M10	行列の書き方 3 通りと、積・逆行列・累乗	[1, 2; 3, 4]、[[1, 2], [3, 4]]、[1 2; 3 4] がどれも [1 2; 3 4]。積 [19 22; 43 50]、逆行列 [−2 1; 3/2 −1/2] = [−2 1; 1.5 −0.5]、2 乗 [7 10; 15 22]、2 × A + 1 = [3 5; 7 9]	合格
M11	det・transpose・adj・rk・rref・identity	det = −3、転置 [1 4; 2 5; 3 6]、adj [4 −2; −3 1]、rk([1, 2; 2, 4]) = 1、rref [1 0 4/5; 0 1 7/5]、identity(3)、A⁻¹ × A = [1 0; 0 1]	合格
M12	連立方程式（2 通り）・固有値・文字を含む行列	[2, 1; 1, 3]^−1 × [3; 5] = [4/5; 7/5]、multisolve で [4/5 7/5]、solve(det([2, 1; 1, 2] − x × identity(2)) = 0) = [3 1]、[1, 2; 3, 4] × [x; y] = [(x + 2y); (3x + 4y)]	合格
M13	ベクトルと、計算できない式のエラー	dot = 32、cross = [0 0 1]、norm([3, 4]) = 5、[1, 2, 3] + 2 × [4, 5, 6] = [9 12 15]。ベクトル × ベクトルは cross()・dot() などを使うよう求めるエラー、[1, 2; 2, 4]^−1 は「Inverse of singular matrix.」、大きさの合わない積もエラー	合格

ID	内容	確かめたこと（実際の値）	結果
M14	trace()（橋渡しプログラムが追加）	trace([1, 2; 3, 4]) = 5、trace([1/2, x; y, 1/3]) = 5/6、trace([1 m, 0; 0, 2 m]) = 3 m、trace(identity(4)) = 4、trace([1, 2, 3; 4, 5, 6; 7, 8, 10]) + det([1, 2; 3, 4]) = 14。正方行列でないと「must be a square matrix.」のエラー	合格
M15	trace() の登録	関数の一覧の Matrices & Vectors に 1 件（題 Trace。利用者の定義ではない）、関数は全部で 425。trace という名前の利用者の関数は作れない。trace を使う利用者の関数 tr2 は作れ、tr2([4, 0; 0, 6]) = 5。定義ファイルに trace は書かれない	合格
M16	ans に行列が入る	[1, 2; 3, 4]^−1 を確定すると det(ans) = −1/2、ans × [1, 2; 3, 4] = [1 0; 0 1]。ans イベントの挿入用の文字 [−2 1; 1.5 −0.5] を計算し直すと同じ行列	合格
M17	導関数と、上端が変数の積分のグラフ	diff(x^3 − 3x)、integrate(2y, 0, x, y)、integrate(e^(−y^2), 0, x, y) の 3 本。x = −2、−1、0、1、2 で 9, 0, −3, 0, 9 と 4, 1, 0, 1, 4 と −0.882081, −0.746824, 0, 0.746824, 0.882081	合格
M18	表示の設定ごとの行列（qalc と同じ表示）	厳密：[−2/3 1/3; 1/2 −1/6] = [−2/3 1/3; 0.5 −1/6]。近似と自動：… ≈ [−0.6666666667 0.3333333333; 0.5 −0.1666666667]。16 進数：[0xA 0xB; 0xC 0xD]	合格
M19	エンジンを起動し直したあと	trace([1, 2; 3, 4]) = 5、利用者の関数 tr2([4, 0; 0, 6]) = 5	合格

日本の単位（11 件すべて合格。版 1.4.0 でやり直し）

橋渡しプログラムが加えた 10 個の単位（詳細設計書 3.19）を確かめました。

ID	内容	確かめたこと（実際の値）	結果
N01	坪と平方メートル	30 坪 to m² = 99.17355372 m²、100 m² to 坪 = 30.25 坪	合格
N02	ローマ字の tsubo、1 坪 = 36 平方尺	1 tsubo to m^2 = 3.305785124 m²、1 坪 to 尺^2 = 36 尺²	合格
N03	長さ：尺・間・寸	6 尺 = 181.8181818 cm、1 間 = 6 尺、10 寸 = 1 尺	合格
N04	里	1 里 = 3.927272727 km、2 km = 0.5092592593 里	合格
N05	体積：合・升	3 合 = 541.1720511 mL、1 升 = 10 合、1.8 L = 0.9978342357 升	合格
N06	質量：貫・匁	1 貫 = 3.75 kg、8 匁 = 30 g、60 kg = 16 貫	合格
N07	反 = 300 坪	2 反 = 600 坪、1 反 = 9.917355372 a	合格
N08	単位を使った計算	3 坪 + 2 m^2 = 11.91735537 m²、80 m² / 坪 = 121/5 = 24.2、坪あたり 35 万円で 25 坪 = ¥8750000	合格
N09	単位の一覧に独自の分類（Japanese Units）で載る。利用者の定義ではない	坪・反・寸・尺・間・里・合・升・匁・貫	合格
N10	名前の保護、起動し直したあとの利用者の変数	坪という名前の変数は作れない（「坪」は組み込みの名前）。madori = 6 坪 は起動し直しても 19.83471074 m²。単位は利用者のファイルに書かれない	合格
N11	短いローマ字は日本の単位として読まない	tan(45°) + 1 = 2（tan は正接のまま）。go・sun も合・寸とは読まない	合格

式の例（73 式すべて計算できる。版 1.4.0 の日本語と英語の画面の式で）

アプリに収めた式の例 69 個と、最初の画面の例 4 個を、画面に書いてあるとおりの文字（×、−、√、π など）でエンジンに送りました。アプリと同じように、プレビュー（制限時間 1.5 秒）と確定（store）の 2 回ずつ計算し、どちらも結果が出ること、エラーと時間切れがないこと、プレビューと確定の結果が同じこと、プレビューが実行を保留しない（deferred にならない）ことを確かめ、73 式すべてが条件を満たしました。

版 1.3.0 では、英語の画面で変わる 4 式（最初の画面の 100 USD to EUR、通貨の例の 100 USD to EUR、50 EUR to GBP、1 BTC to USD）を含めて、日本語と英語の画面の式（各 73 式）を 1.3.0 のエンジンで計算し、どれもエラーなく結果が出て、プレビューと確定の結果が同じでした。

メモリの検査とランダムな要求

試験	内容	結果
機能試験（検査用のビルド）	E01〜E24（26 件）、N01〜N11、M01〜M19 を、メモリの検査を入れたエンジンで実行	N・M はすべて合格。E は 25/26 で、検査の報告は 0 件。不合格の E22 は、時間切れの応答（4.2 秒）から 1 秒たってもプロセスが終わっていなかったため。同じ要求を検査用のビルドで改めて試すと、4.6 秒で時間切れを応答し、その 0.1 秒後に終了コード 3 で終わった（合格の条件を満たす）
ランダムな要求（検査用のビルド）	命令（不正なものを含む 16 種類）、式（壊れた式・巨大な式・深い入れ子、解析と行列のひな形、trace() など約 80 種類）、型の違う値、壊れた JSON（途中で切れた行、余分な文字、ランダムなバイト列、深い入れ子、不正な文字）を乱数で組み合わせた要求。11 通りの乱数で 3,000 件ずつ、計 33,000 件	すべてに応答。エンジンの異常終了 0 回、検査の報告 0 件（標準エラー出力 0 バイト）
ランダムな要求（配布するエンジン、qemu）	同じ作り方の要求を、4 通りの乱数で 1,500 件ずつ、計 6,000 件	すべてに応答。時間切れによる起動し直しが 2 回（終了コード 3。想定どおりの動き）。それ以外の終了はなし

高精度の計算（16 件すべて合格。版 1.2.0）

精度を上限の 200000 桁にして、配布するエンジン（arm64）を PC 上で模擬実行しました。結果の全部の桁は、別のライブラリ（Python の mpmath 1.4.1）で同じ桁数に丸めた値、または PC 向けに作った公式の qalc の結果と照合しています。時間は模擬実行のもので、PC で直接動かすとおよそ 10 分の 1 です。

ID	内容	結果
H01	π、e、√2、∛2 を 200000 桁で：各 200,001 文字。mpmath の値と全部の桁が一致（模擬実行で 1.5〜3.8 秒）	合格
H02	sin(1)、ln(2)、cos(π/7) を 200000 桁で：mpmath の値と全部の桁が一致（模擬実行で 7.7〜9.5 秒。PC の qalc では sin(1) が 1.2 秒）	合格
H03	atan(1)、exp(1)、1/7、√(−2) + π を 200000 桁で：PC の qalc と同じ（複素数は 400,006 文字）	合格
H04	精度 100000 桁でも、π、e、√2、sin(1)、ln(2)、∛2、cos(π/7) が mpmath と全部の桁で一致。atan(1)、exp(1)、π × 10^−5 は qalc と一致	合格
H05	10000! は 35,660 桁すべて（0.02 秒）	合格
H06	精度 300000 を求めると 200000 桁で計算する（上限に収める）	合格
H07	精度 1000 と 10 は今までどおり（π が 1,001 文字と 11 文字）	合格
H08	3 桁区切りを付けた 200000 桁の π（266,667 文字、約 40 万バイト）は切られない	合格
H09	450,000 バイトを超える結果（[π, e; √2, √3]）は末尾を切って「…」、truncated:true	合格
H10	入力中の表示（max_chars 6000）は 6,000 バイトで切って「…」	合格
H11	精度 200000 のままでも 2D グラフは 20 桁で計算して 0.07 秒（3D グラフは精度 100000 で 0.11 秒）。そのあとの π は全部の桁（精度が戻る）	合格
H12	ζ(3) の入力中の表示（1.5 秒）は、ウォッチドッグが 4.6 秒で時間切れを返し、エンジンを起動し直す	合格
H13	精度 1000 桁の ans を新しいエンジンに戻すと 1,001 文字のまま（1.1.0 では約 38 桁に丸まっていた）	合格
H14	200000 桁の ans（π の値は 400,076 文字）を新しいエンジンに戻す：模擬実行で 1.2 秒（GMP で読む前は 12.5 秒）。ans と ans × 2 は元と同じ。ln(2) と複素数（800,155 文字、2.6 秒）も同じ	合格

ID	内容	結果
H15	長い数を GMP で読んだ結果は、libqalculate が読んだ結果と同じ：16 の値（正・負、指数 ±、複素数、行列、単位付き、分数、200000 桁など）で、ans・ans × 3 − 1・保存し直した値がすべて一致。200000 桁の π は 0.18 秒（libqalculate だけでは 3.40 秒。PC で直接実行）	合格
H16	精度 100000 桁のほかの設定：厳密モードの √2 は記号のまま、小数表示の 1/3 は 100,002 文字、π to hex は 83,051 文字、100000! は 100,008 文字	合格

版 1.3.0 で変わった点：H09 と H10 は版 1.2.0 の動きです。版 1.3.0 では、450,000 バイトを超える結果は先頭と末尾を残して途中を省き（tail_chars。「長い結果のため、途中を省略しています（全 N 桁）」）、入力中の表示も 450,000 バイトまで出します（詳細設計書 3.5・3.8・3.9）。Q02 で確かめています。

6.3 Android 部分（Java）のテスト項目と結果

本物のエンジンを使う試験（21 件すべて合格。版 1.4.0 のエンジンでやり直し。Engine.java と StateStore.java は版 1.3.0 から変えていない）

Engine が qemu で動くエンジンを起動して使います。

ID	内容	結果
J01	init が成功する（約 0.23 秒）	合格
J02	1/3 を確定すると 0.3333333333	合格
J03	同じエンジンで ans × 3 = 1	合格
J04	ans イベントの値（1/3）を SharedPreferences に保存	合格
J05	エンジンを止めたあとの要求で起動し直し、ans × 6 = 2（ans が戻っている）	合格
J06	計算中の中止：fibonacci(10^9) を 1.5 秒後に cancel すると code=cancelled（1.505 秒）	合格
J07	中止のあとも 2+2 = 4	合格
J08	null の要求は code=bad_request	合格
J09	改行を含む要求は code=bad_request	合格

ID	内容	結果
J10	JSON でない要求は code=bad_request	合格
J11	rand() の結果が、エンジンを起動し直したあとの ans と一致	合格
J12	StateStore：書き込み待ちの内容を読める	合格
J13	StateStore：state/qalc.history.json ができる	合格
J14	StateStore：「../evil」のような名前は拒否	合格
J15	版 1.4.0 のエンジンで init が成功する	合格
J16	精度 200000 桁で π を確定すると 200,001 文字	合格
J17	その ans の値（interval(下限, 上限)、400,076 文字）を SharedPreferences に保存する（500,000 文字まで）	合格
J18	エンジンを止めたあとの要求で起動し直し、ans が全部の桁で戻る（π と同じ 200,001 文字。起動・復元・計算で約 2.5 秒、PC 上の模擬実行）	合格
J19	精度 1000 桁の √2 も、起動し直したあと 1,001 文字のまま（1.1.0 では約 38 桁に丸まっていた）	合格
J20	450,000 バイトを超える結果（[π, e; √2, √3]）は切られ、末尾が「…」	合格
J21	500,000 文字を超える ans の値（同じ行列。約 80 万文字）は保存しない（null）	合格

試験用の偽エンジンを使う試験（10 件すべて合格。版 1.4.0 でやり直し）

応答しない、途中で終わる、壊れた行を出すなど、本物では起こしにくい動きをする小さなプログラムをエンジンの代わりに使います。

ID	内容	結果
F01	init が成功する	合格
F02	応答しないエンジンは待ち時間（timeout 500 + 10 秒）で code=timeout（10.501 秒）	合格
F03	次の要求で新しいエンジンが起動して応答する	合格
F04	関係のない行と途中で切れた行を読み飛ばす	合格

ID	内容	結果
F05	要求のあとに終了したら code=crash（送り直さない）	合格
F06	そのあとの要求は成功する	合格
F07	restart=true の応答をそのまま返す	合格
F08	そのあとの要求は新しいエンジンで成功する	合格
F09	ans イベントを画面へ渡し、値を保存する	合格
F10	イベントの行を応答と取り違えない	合格

MainActivity（WebView、戻るボタン、表示処理の停止からの復帰）と RatesUpdater の通信は Android の環境が必要なため、この試験には含まれません（6.6）。

版 1.3.0 で加えた Java（Pro.java、MainActivity の saveFile と tr、PlayBilling.java）も Android の環境が必要なため、この試験には含まれません。PlayBilling.java は、Google Play Billing Library 8 の型を模した代用クラスに対してコンパイルできることだけを確かめています（課金のある版のビルドと購入は 6.6）。Pro の状態に応じた画面の動きは、Java の代わりの模擬で画面の試験（Q04〜Q18）として確かめました。

6.4 画面のテスト項目と結果

画面を Chromium で開き、試験用のスクリプトがキー操作などを行って結果を確かめました（計算は本物のエンジン）。

キーのタブ・解析・行列の画面（26 件すべて合格。版 1.4.0 の日本語の画面でやり直し）

版 1.1.0 で変えた画面（キーのタブ、ひな形、行列の表、式の例）を、いくつかの画面の大きさで確かめました。

ID	内容	実際の結果	結果
W01	起動したときのキー	タブ 4 つ（「123」が選ばれている）、キー 25 個、上段のキー 8 個（x , a=b → ans ⌨ ◀▶）	合格
W02	360×740 で最初の画面が欠けずに見える	説明の上端 14 px、下の余白 5 px、スクロールなし。式の例 4 つ	合格

ID	内容	実際の結果	結果
W03	タブでキーが替わる	関数（sin cos tan …）、解析（d/dx d²/dx² ∂/∂y …）、行列（[ ] ; …）、123（( ) % …）。どれも 25 個で、選んだタブの色が変わる	合格
W04	∫ₐᵇ のひな形	integrate(, 0, 1) が入り、カーソルは「(」の直後。x^2 を入れると integrate(x^2, 0, 1) = 1/3 ≈ 0.3333333333	合格
W05	ひな形のカーソル位置と ⌫	∫ₐᵇ・d²/dx²・∂/∂y・lim・Σ・Π・d/dx・極値など、すべて最初の引数の位置。中身のないひな形は ⌫ 1 回で消える	合格
W06	d²/dx² と ∂/∂y	diff(x^5, x, 2) = 20x³、diff(x^2 × y^3, y) = 3x²y²	合格
W07	lim・Σ・Π	limit(sin(x)/x, 0) = 1 と「limit() は完全ではなく、結果が正しくない場合があります」、sum(x^2, 1, 10) = 385、product(x, 1, 5) = 120	合格
W08	dsolve のひな形、a=b キー、∞ キー	dsolve(diff(y, x) =) でカーソルは最後の「)」の前。dsolve(diff(y, x) = y) = e^x × C …、a=b を使った solve(x^2 = 16) = [4 −4]、limit((1 + 1/x)^x, ∞) = e ≈ 2.718281828	合格
W09	選んだ文字をひな形で囲む	x^2 × sin(x) を選んで ∫ₐᵇ → integrate(x^2 × sin(x), 0, 1)、−2y を選んで dsolve → dsolve(diff(y, x) = −2y)	合格
W10	「ほか…」	解析では関数の一覧が「微積分」（18 件）、行列では「行列・ベクトル」（51 件、trace を含む）を選んだ状態で開く。「挿入」で limit() が入り、一覧が閉じる	合格
W11	行列のキーで [1, 2; 3, 4] と A⁻¹	式は [1, 2; 3, 4]^(−1)。プレビューに 2 列の表が 2 つ（−2、1、3/2、−1/2 と −2、1、1.5、−0.5）。読み上げ用の名前は [−2 1; 3/2 −1/2]	合格
W12	履歴の行列と、結果のタップ	履歴に表が 2 つ。結果をタップすると [−2 1; 1.5 −0.5] が文字で入り、計算すると同じ行列	合格

ID	内容	実際の結果	結果
W13	det・A^T・I_n	det([1, 2; 3, 4]) = −2、transpose([1, 2, 3; 4, 5, 6]) は 3 行 2 列の表、identity(3)	合格
W14	行列の積、幅の広い行列	積 [19 22; 43 50]。3×3 の逆行列の近似値（9 要素）は文字を 0.738em に縮めて画面に収まる	合格
W15	行ベクトルは文字のまま、列ベクトルは縦の表	[1, 2, 3] + 2 × [4, 5, 6] = [9 12 15] は表にしない。連立方程式の解 [4/5; 7/5] は縦に並べる	合格
W16	trace() と日本語の検索	trace([1, 2; 3, 4]) = 5。関数の一覧を「トレース」で検索すると 1 件（trace(Matrix)、行列・ベクトル）	合格
W17	無限大を端にもつ積分のあとの警告	積分のあとの 1/0 に「ゼロで割っています」。積分と 1/0 を含む式（= 1 + 1 / 0）でも同じ	合格
W18	式の例の分野	12 分野（微分、積分、極限・級数・微分方程式、行列、ベクトルを含む）	合格
W19	⌨	タブとキーを隠し、もう一度押すと同じタブ（行列）で戻る	合格
W20	縦向きでスクリプトのエラー	0 件	合格
W21	ダークテーマと文字の大きさ「特大」	キーは画面内（下の余白 8 px）、行列も表示する場所に収まる	合格
W22	ダークテーマでスクリプトのエラー	0 件	合格
W23	横向き 740×360	タブ（高さ 26 px）と 5 段のキー（高さ 42 px）が収まる	合格
W24	横向きでスクリプトのエラー	0 件	合格
W25	320×568	キーは画面内、最初の画面は先頭から表示（スクロール位置 0）	合格
W26	x^3 − 3x とその導関数のグラフ	2 本を描き、前のグラフでタップした点の表示は消える	合格

式の例・元に戻す・文字の大きさ・検索（22 件すべて合格。版 1.4.0 の日本語の画面でやり直し）

ID	内容	実際の結果	結果
V01	最初の画面の式の例	60 mph to km/h、100 USD to JPY、solve(x^2 − 5x + 6 = 0)、diff(x^3)	合格
V02	例をタップ	入力欄に入り、プレビューに = 96.56064 km/h	合格
V03	式の例のパネル	題は「式の例・使い方」、12 分野・69 個、ヒント 12 個（版 1.4.0 で候補の説明を 1 個追加）	合格
V04	例で式を置き換える	30 坪 to m² が入り、パネルが閉じ、「元に戻す」が出る	合格
V05	元に戻す	置き換える前の 60 mph to km/h に戻る	合格
V06	AC のあとの元に戻す	入力欄が空になり、「元に戻す」で戻る	合格
V07	AC のあとに入力する	「元に戻す」が消える	合格
V08	履歴の式のタップ	入れ替わった式（1+2）を元に戻せる	合格
V09	履歴の行の削除	3×4 を削除すると、通知「1 行を削除しました」と「元に戻す」ボタン。戻すと元の位置に戻る	合格
V10	履歴の消去	消去すると最初の画面（例 4 つ）。「元に戻す」で 1+2、3×4、5×5 が戻る	合格
V11	文字の大きさ「特大」	結果 22 → 30.36 px、入力 26 → 35.88 px（1.38 倍）、キーは 17 px のまま。設定は 20 項目（版 1.4.0）で、保存される	合格
V12	起動し直したあとの文字の大きさ	特大のまま（30.36 px）。標準に戻せる	合格
V13	関数を日本語で検索	全部で 421 件。「積分」で Integrate（積分）が先頭、「最小」と「min」で Min（最小値）が上位	合格

ID	内容	実際の結果	結果
V14	単位の一覧の「日本の単位」	10 個（貫・間・合・尺・升・寸・坪・反・匁・里）。km と坪を日本語の名前で探せる	合格
V15	単位の「変換」ボタン	「to 坪」が入る	合格
V15b	複合単位の表示と挿入	km/h と表示し、挿入した 90km/h は 90 kilometers/hour と解釈される	合格
V16	定数を日本語で検索、変数名 constructor	「光速」で c（Speed of Light in Vacuum）。constructor = 5 という利用者の変数も普通に表示	合格
V17	メニュー	13 項目（版 1.3.0 で QCalc Pro、Language / 言語、履歴の書き出しを追加）で、先頭は「式の例・使い方」。「このアプリについて」からも式の例を開ける	合格
V18	履歴が空のときの「履歴を消去」	「履歴はありません」と表示するだけで、確認を出さない	合格
V19	読み上げ	メッセージは role=status、「元に戻す」は button	合格
V20	ダークテーマ・横向きの最初の画面	横にはみ出さない（幅 854 px）	合格
V21	ページのエラー	0 件	合格

履歴の再利用と入力の変換（12 件すべて合格。版 1.4.0 の日本語の画面でやり直し）

ID	内容	実際の結果	結果
U01	7/3 の結果をタップ	(7/3) を挿入	合格
U02	255 to hex（0xFF）の結果をタップ	255 を挿入	合格
U03	x^2 = 4 の結果をタップ	[2 −2] を挿入	合格

ID	内容	実際の結果	結果
U04	2x + 3 = 7 の結果をタップ	2 を挿入	合格
U05	5 ft to m の結果をタップ	(1.524 m) を挿入	合格
U06	sqrt(2) の結果をタップ	1.414213562 を挿入	合格
U07	1/0.25 の結果をタップ	4 を挿入	合格
U08	「2×」のあとに 7/3 の結果を挿入して確定	= 14/3 = 4 + 2/3 ≈ 4.666666667	合格
U09	全角の「１０＋５」を入力	10+5 に変換	合格
U10	「100円」を入力	100¥ に変換	合格
U11	100¥ を確定	= ¥100	合格
U12	q = 5 を入力中	「= を押すと実行します（定義など）」と表示し、実行しない	合格

操作と表示（9 件すべて合格。版 1.4.0 の日本語の画面でやり直し）

ID	内容	実際の結果	結果
U13	キーへの普通の click（画面読み上げやキーボードの操作）	1 回だけ入力	合格
U14	タッチのあとに click が続いても	1 回だけ入力	合格
U15	2,600 文字のプレビューでもキーが画面内に残る	= キーの下端 846 / 画面 854	合格
U16	x + 1e17 のグラフ	固まらずに描画	合格
U17	グラフを 60 回拡大	固まらない	合格

ID	内容	実際の結果	結果
U18	単位の一覧で「円」を検索	Japanese Yen が見つかる	合格
U19	関数の一覧で「command」を検索	使えない command() は表示しない（0 件）	合格
U20	計算中に入力した文字	確定のあとも消えずに残る（123）	合格
U21	1/0 のメッセージ	「ゼロで割っています」	合格

3D グラフ（10 件すべて合格。版 1.4.0 でやり直し）

グラフの画面を「3D」に切り替えて操作し、表示を画像でも確かめました（表示サイズ 405×720）。

ID	内容	実際の結果	結果
G01	3D に切り替えて sin(x) cos(y) を描く（−5〜5、標準 41×41）	面と軸、色の帯（z：−1.00〜1.00）を表示。メッセージなし	合格
G02	ドラッグで回転（横、縦）	向きが変わり、軸と目盛りも一緒に回る	合格
G03	面をタップ	「sin(x) cos(y)：x = 1.5, y = -1.25, z = 0.3145324757」と赤い点	合格
G04	極のある面 1/(x^2 + y^2)（−2〜2）	極の近くの大きすぎる値を色の範囲から外して描く（z：0.1〜23.0）	合格
G05	波紋 sin(sqrt(x^2 + y^2)) / sqrt(x^2 + y^2)（−10〜10、細かい 61×61）	中央の山と輪を描く	合格
G06	2 つの面 x^2/10 − y^2/10 と 2（粗い 25×25）	2 色の面と凡例	合格
G07	sqrt(x)（x < 0 は実数にならない）	x ⩾ 0 の部分だけを描く（z：0〜2.24）	合格
G08	y の範囲に誤り	「y の範囲を確認してください」	合格
G09	2D に戻してグラフを閉じ、開き直す	2D のまま開く（最後のモードを覚えている）	合格

ID	内容	実際の結果	結果
G10	ダークテーマと横向き	崩れずに表示。コンソールのエラー・警告 0 件	合格

見直しで直した点の確認（10 件すべて合格。版 1.4.0 でやり直し）

コードの見直しで直した点が、直ったままであることを確かめました。

ID	内容	実際の結果	結果
R01	「a𝑥b」の末尾で ◀	カーソルが 4 → 3	合格
R02	続けて ◀（𝑥 はサロゲートペアで 2 文字分）	1 回で 3 → 1（文字の途中で止まらない）	合格
R03	▶	1 回で 1 → 3	合格
R04	範囲を選んで ◀	選択の先頭（1）にカーソル	合格
R05	範囲を選んで ▶	選択の末尾（3）にカーソル	合格
R06	2D グラフで 1/(x−x)（0 で割る）	英語のメッセージ（Division by zero）をそのまま出さない	合格
R07	3D で実数の値がない sqrt(−1 − x^2 − y^2)	凡例に「(値なし)」、色の帯は出さず、「実数の値がありません（x と y の範囲を確認してください)」	合格
R08	グラフの canvas に読み上げ用の名前	「2D グラフ」「3D グラフ」	合格
R09	61×61 の面 2 つを 60 フレーム回転	1 フレーム 20 ms（この PC。版 1.2.0 では 21 ms）で、固まらずに回る	合格
R10	ページのエラー	0 件	合格

長い結果の表示と保存（32 件すべて合格。版 1.2.0。版 1.4.0 の日本語の画面でやり直し）

精度を 200000 桁にして、画面を Chromium（幅 412・高さ 915 の画面）で動かしました。計算エンジンは、配布するもの（arm64、PC 上で模擬実行）と PC 向けのメモリ検査付きのものの両方で試し、どちらも 32 件すべて合格しました。このほか、履歴の再利用（12 件）と操作と表示（9 件）の試験も 1.2.0 の画面でやり直し、すべて合格しました。

ID	内容	結果
L01〜L02	設定の「有効桁数」の説明が「2〜200000桁」、入力欄の上限が 200000	合格
L03〜L06	300000 を入れると 200000、1 を入れると 2 に直る。1000 と 200000 はそのまま。どれも保存される	合格
L07	「200000」が入力欄に収まる	合格
L08〜L09	π を確定すると全部の桁（200,003 文字。「≈ 」を含む）が出て、末尾も正しい	合格
L10〜L11	長い結果は枠の中でスクロールし、枠の高さは履歴の 70 % まで（275 / 393 px）	合格
L12	キーが画面から押し出されない	合格
L13	読み上げ用の文は先頭だけ（204 文字）	合格
L14	長押しメニューの見出しは先頭だけ（304 文字）	合格
L15	「結果をコピー」で 200,001 文字すべてがクリップボードに入る	合格
L16	20 万桁の結果はタップしても式に入らず、「ans キーで使えます」と出る	合格
L17	入力中の ans × 2 は先頭 6,000 文字と「…」	合格
L18	ans × 2 を確定すると全部の桁	合格
L19	ページを読み込み直しても、2 つの結果が全部の桁で戻る	合格
L20〜L25	20 万桁の結果を 12 件にすると、新しい 5 件は全部の桁、残りの 7 件は先頭 2,000 文字と「…」。保存した文字は 1,015,339 文字で、画面も同じ	合格
L26〜L27	短い結果（1 + 2 = 3）は今までどおりで、枠のスクロールは出ない	合格
L28	複素数 √(−2) + π（実部と虚部が 20 万桁、400,008 文字）も全部表示	合格

ID	内容	結果
L29〜L30	3 桁区切りを付けた π（266,669 文字）を全部表示し、266,667 文字すべてをコピーできる	合格
L31	精度を 10 に戻すと π ≈ 3.141592654	合格
L32	スクリプトのエラーがない	合格

版 1.3.0 で変わった点：L17（入力中の ans × 2 は先頭 6,000 文字）は、入力中も全部の桁を出すようになったため Q02 に、L29〜L30（3 桁区切りの π）は、小数部にコンマを付けず狭い空白で区切るようになったため Q03 に置き換えました。版 1.4.0 では、32 項目を日本語の画面でやり直し、すべて合格しました。ただし 2 項目は、今の動きに合わせて確かめ方を変えています。L14 の見出しは、版 1.4.0 から計算した日時で始まり（日時、式、結果の先頭の順で 322 文字。T19）、L17 は Q02 と同じく、入力中も全部の桁（200,003 文字）が出ることを確かめました。

版 1.3.0 の画面（27 件すべて合格。版 1.4.0 でやり直し）

版 1.3.0 で加えた機能と変えた動きを、Chromium（412×915）で確かめました。計算は同じソースから PC 向けに作ったエンジン、QCalc Pro の購入は Java の代わりの模擬（billing の種類と購入の状態を切り替える）です。Q01〜Q19 は QCalc Pro・配色テーマ・書き出し・プレビュー・桁区切り、Q20〜Q26 は表示の言語、Q27 は英語の画面の確認です。

ID	内容	実際の結果	結果
Q01	最初の起動	英語の画面。メニューの先頭は「Examples and tips」で、「QCalc Pro」（テスト版）と「Export history (CSV)」がある	合格
Q02	4587! のプレビュー	プレビューも履歴と同じ 14,806 桁すべて（末尾の 0 が 1,144 個）	合格
Q03	桁区切り	1,234,567。12.345 km（単位の前にコンマを付けない）。3.141 592 654（小数部にコンマを付けない）	合格
Q04	テスト版の Pro の画面	「QCalc Pro is unlocked」、機能 5 つ、PRO の印なし	合格
Q05	配色テーマ「Green」	data-theme が green、強調の色 #15803d、保存される	合格

ID	内容	実際の結果	結果
Q06	履歴の書き出し	UTF-8（BOM 付き）、見出しと 4 行。数値は区切りなし、式として実行される欄がない	合格
Q07	Pro なし：PRO の印	「Export history」に PRO の印	合格
Q08	Pro なし：配色テーマ	「Color themes are a QCalc Pro feature」と出てブルーのまま	合格
Q09	Pro なし：有効桁数 5000	1000 に戻り、「A precision above 1000 digits is a QCalc Pro feature」	合格
Q10	Pro なし：Pro の画面	価格付きの「Buy QCalc Pro (¥300)」と「Restore a purchase」	合格
Q11	購入がないときの復元	「No purchase of QCalc Pro was found」	合格
Q12	購入したあと	「QCalc Pro is unlocked. Thank you!」、Pro の画面は使える表示、PRO の印が消える	合格
Q13	購入したあとの 3D	3D グラフが開く	合格
Q14	Pro なし：3D	「3D graphs are a QCalc Pro feature」と出て 2D のまま	合格
Q15	Pro なし：書き出し	「Exporting the history is a QCalc Pro feature」	合格
Q16	課金のない版	「QCalc Pro」の項目がなく、配色テーマ（Purple）も使える	合格
Q17	日本語の Pro の画面	「QCalc Pro を購入（¥300）」、機能の先頭は「高精度」、4 番目は「履歴の書き出し」	合格
Q18	ダークテーマとオレンジ（購入後、日本語）	強調の色 #fb923c	合格
Q19	ページのエラー（Q01〜Q18）	0 件	合格
Q20	最初の起動	英語。履歴 1 件と入力中の 7*6	合格

ID	内容	実際の結果	結果
Q21	メニューで「日本語」	日本語の画面になり、履歴と 7*6 が残る	合格
Q22	設定で「English」（設定の最初の項目）	英語に戻り、履歴と 7*6 が残る	合格
Q23	ページのエラー（Q20〜Q26）	0 件	合格
Q24	テスト版が設定に lang "ja" を残し、qalc.lang がないとき	英語で始まる（ほかの設定は残る）	合格
Q25	メニューで日本語を選んだあと、開き直す	日本語のまま（qalc.lang が "ja"）	合格
Q26	もう一度英語を選ぶ	英語のまま（qalc.lang が "en"）	合格
Q27	英語の画面に日本語が残っていない	最初の画面、4 つのキーのタブ、メニュー、10 のパネル（3D とライセンスを含む）、5 つの計算の結果と履歴を調べ、日本語は「Language / 言語」「日本語」と日本の単位の名前（坪など）だけ	合格

版 1.4.0 の画面（27 件すべて合格。版 1.4.0）

版 1.4.0 で加えた機能と直した動きを、Chromium（412×915。T24 は横向き 780×360）で確かめました。計算は同じソースから PC 向けに作ったエンジンです。T01〜T24 と T27 は英語の画面、T25・T26 は日本語の画面です。端末のキーボードでの入力は、入力欄に input イベントを送って模擬しています。

ID	内容	実際の結果	結果
T01	キーボードで「sin」と入力	候補の先頭に sin()、sinh()、sinc()	合格
T02	関数の候補をタップ	名前とかっこが入り、カーソルはかっこの中。候補は消える	合格
T03	式の途中で「integ」と入力して候補をタップ	入力した文字だけが integrate() に置き換わる（2 + integrate()）	合格

ID	内容	実際の結果	結果
T04	1文字だけのとき、変数や単位を全部書いたとき（pi、kg）	候補は出ない	合格
T05	画面のキーで sin を入力	名前の候補は出ない	合格
T06	「pound」と入力	題の中の語で lb と GBP が見つかる	合格
T07	カーソルを名前から離す（◀）	名前の候補が消える	合格
T08	「5 km」のあと →	同じ種類の単位が、よく使うもの（m、cm、mm、mi、ft）から並ぶ。km は出ない	合格
T09	変換先の候補 mi をタップ	「5 km to mi」になり、プレビューに変換した結果が出る。候補は消える	合格
T10	「100 USD」のあと →（英語の画面）	EUR、JPY、GBP の順。USD は出ない	合格
T11	単位のない「255」のあと →	fraction、hex、bin、oct の順。hex をタップすると「255 to hex」で 0xFF	合格
T12	°C、km/h、接頭辞を付けた kPa、名前で書いた hours	それぞれ °F・K、m/s・mph・knot、Pa・bar・atm、s・min・d が先頭	合格
T13	種類の違う単位がまじった式（5 km / 2 h）、関数（sin(30°)）	前者は候補なし。後者は sin を単位と読まず、数の表し方が出る	合格
T14	「to」の後に文字を入力	「5 km to m」で mm・mi など同じ種類だけ、「255 to he」で hex。単位を全部書いたとき（30 tsubo to m²）は、別の種類の単位（m²·A など）を出さない	合格
T15	日本の単位「30 坪」のあと →	面積の単位 m²、km²、ha が先頭	合格
T16	式の終わりが「to」だけ（255 to、5 km to）	プレビューは「to」の前までの結果（= 255、= 5 km）	合格
T17	「7 km to」で ＝	「7 km」を計算して履歴に残す	合格

ID	内容	実際の結果	結果
T18	続けて計算する	新しい 1 件だけが履歴の表示に足され、前の行は作り直されない。行列は表の形で出る	合格
T19	履歴の行を長押し	メニューの見出しの先頭に計算した日時、続いて式と結果	合格
T20	行を削除して「元に戻す」	選んだ行が消え、同じ位置に戻る	合格
T21	履歴が 300 件のときに計算	いちばん古い 1 件が消え、新しい結果が出る（300 件のまま）	合格
T22	設定「ライト／ダーク」	ダークとライトは端末の設定に関係なくその配色、「端末の設定と同じ」は端末に合わせる	合格
T23	端末がダークのとき「ライト」を選んで開き直す	ライトのまま	合格
T24	横向きで長い式を入力	キーと上部のボタンが画面の中に収まる	合格
T25	日本語の画面	「積分」で integrate()。通貨の候補は JPY が先頭。長さの候補は、よく使う単位のあとに日本の単位（寸など）	合格
T26	日本語の画面の候補	説明は日本語の名前。読み上げ用の名前は「候補」	合格
T27	有効桁数 300 の π と、x・y・z の 14 項の多項式	長い数は「≈」と同じ行から始まり、行の終わりで折り返す。長い数のない結果（多項式）は語の間で折り返す	合格

T01〜T27 の間、ページのスクリプトのエラーは 0 件でした。

6.5 APK と App Bundle の検証項目と結果

検査したファイル：テスト用 APK「out/QCalc-1.4.0-debug.apk」（3,319,892 バイト、SHA-256 4e50a94b…966a）と、Google Play 用 App Bundle「out/QCalc-1.4.0.aab」（3,310,192 バイト、SHA-256 b7731e62…04e5）。中の計算エンジンは 7,588,488 バイト（SHA-256 ac8f1bd9…b9a7）。K01〜K10 は主にテスト用 APK とエンジン、K11〜K22 は主に App

Bundle と、そこから bundletool で作った端末向けの APK です。bundletool はダウンロードしたものなので、ネットワークから切り離した環境で実行しました。版 1.4.0 の 2 つのファイルで 22 項目をすべてやり直しました（2026-11-13）。

ID	内容	実際の結果	結果
K01	テスト用 APK の署名（apksigner verify）	v1・v2・v3 で検証成功（Android 6 以降の検査を含む）	合格
K02	整列（zipalign -c -p 4）	検証成功	合格
K03	パッケージ情報（aapt2 dump badging）	io.github.hiroshitsu.qcalc.debug、versionCode 6、versionName 1.4.0-debug、minSdk 24・targetSdk 36、起動するアクティビティ io.github.hiroshitsu.qcalc.MainActivity、arm64-v8a	合格
K04	権限	INTERNET（為替レートの取得）と VIBRATE（キーの振動）だけ	合格
K05	中身が最新	エンジンと画面の 4 ファイル（app.js・index.html・licenses.js・style.css）がソースと同じ	合格
K06	余分なものがない	署名のファイルを除いて 24 項目	合格
K07	エンジンの入れ方	圧縮して入っている（extractNativeLibs により、インストールのときに Android が実行できる形で取り出す）	合格
K08	エンジンの形式	AArch64 の実行ファイル（ELF、EXEC）	合格
K09	16 KB ページ	LOAD セグメント 4 つ、境界はすべて 0x4000	合格
K10	静的リンク	共有ライブラリを必要としない	合格
K11	App Bundle の署名（jarsigner -verify）	検証成功	合格
K12	署名した鍵	アップロード鍵（CN=QCalc upload key, O=HiroshiTsu）。テスト用の鍵ではない	合格

ID	内容	実際の結果	結果
K13	App Bundle と APK の中身が同じ	エンジン、画面のファイル、classes.dex が一致	合格
K14	App Bundle の構成	BundleConfig.pb と base/（manifest、dex、lib/arm64-v8a、assets、res、resources.pb）だけ	合格
K15	App Bundle に鍵やパスワードのファイルがない	なし	合格
K16	APK に鍵やパスワードのファイルがない	なし	合格
K17	bundletool validate	受け付けられる	合格
K18	App Bundle のマニフェスト	io.github.hiroshitsu.qcalc、versionCode 6、versionName 1.4.0、minSdk 24・targetSdk 36	合格
K19	App Bundle のマニフェスト（続き）	extractNativeLibs、権限は INTERNET と VIBRATE だけ、debuggable ではない	合格
K20	bundletool build-apks	arm64 の Android 16 の端末向けに APK を作れる	合格
K21	端末に届く APK のエンジン	base-master.apk に同じエンジンが入っている	合格
K22	端末に届く APK の情報	io.github.hiroshitsu.qcalc、versionCode 6、versionName 1.4.0、ラベル QCalc、起動するアクティビティ io.github.hiroshitsu.qcalc.MainActivity	合格

6.6 未実施のテストと既知の制約

まだ行っていないテスト（実機や Android の開発環境が必要なもの）

項目	理由・確かめるときの観点
実機へのインストールと起動（受け入れ基準 A-02）	AQUOS sense8（Android 16）での確認が必要。エンジンが nativeLibraryDir から起動できることも含む。テスト用 APK と、内部テストで Google Play から入れた版の両方で確かめる

項目	理由・確かめるときの観点
TalkBack での操作（A-10）	読み上げの内容と、キー・履歴・メニューの操作
為替レートの実際の取得（A-11）	試験環境からは外部のサーバーに接続できないため、3 か所の取得元からのダウンロードと検査は未確認
端末の日本語キーボードでの入力	自動試験では入力イベントを模擬している。変換中の文字、Enter、キーボードを閉じたときの戻り方
Android System WebView での表示	PC の Chromium との差、文字や配置の崩れ
解析・行列のキーと行列の表の、実機での見え方	Android System WebView での上付き・下付きの文字（$\int_a^b$、A^T、I_n）と表の線。TalkBack では行列を 1 行の文字（[1 2; 3 4]）として読み上げる作り（詳細設計書 3.9）なので、その聞こえ方も確かめる
画面の回転、ダークモードの切り替え、戻る操作（予測型「戻る」を含む）、振動、コピー	MainActivity の処理は PC で動かしていない。Android 15 以降でステータスバー・ナビゲーションバー・キーボードと画面が重ならないことも確かめる
アプリを閉じたあとの復元	履歴・入力中の式・設定・ans が残ること（保存処理そのものは 6.3 で確認済み）
メモリが少ないとき、長時間の使用	表示処理が止まったときの作り直しを含む
Android 7.0〜15 の各版	対応版の下限に近い端末での動作
Android Studio（Gradle）でのビルドと実行	試験環境から Gradle と Android SDK を取得できないため未確認。同期、デバッグ版の実行、release の署名と上書きインストールを確かめる
16 KB ページの端末	エンジンは 16 KB の境界で作ってあり（K09）Google Play の条件を満たすが、16 KB ページの端末では動かしていない
3D グラフの実機での速さ	PC では 61×61 の面 2 つの回転が 1 フレーム 21 ms（R09）。端末の性能によっては回転が遅くなる。細かさを「粗い（25×25）」にすると軽くなる

項目	理由・確かめるときの観点
高精度の計算の実機での速さとコピー（A-19 の実機での確認）	PC では 20 万桁の sin(1) が 1.2 秒、π が 0.4 秒。実機での時間と、20 万桁の結果（3 桁区切り付きで約 27 万文字）のクリップボードへのコピー、アプリを閉じたあとの ans の復元を確かめる
Google Play の審査とクローズドテスト	.aab を Play Console に上げ、事前審査レポートとテスターの端末で確かめる（公開の流れは 5.7）
QCalc Pro の購入（課金のある版）	Gradle（qcalc.billing=true）でのビルドは、試験環境から Google の Maven リポジトリに接続できないため未確認。Play Console にアイテム qcalc_pro を作り、ライセンス テスターのアカウントで、内部テストから入れた版で、購入・復元・支払い待ち・払い戻しと、以前からの利用者（legacy）の扱いを確かめる
表示の言語と履歴の書き出しの実機での確認	英語で始まり、Language / 言語 で切り替えても履歴と入力中の式が残ること。Java が自分で出す文（コピーの通知など）の言語。システムのファイル選択画面での保存（端末と Google ドライブなどの保存先）
入力の候補とライト／ダークの実機での確認（版 1.4.0）	端末のキーボード（日本語入力を含む）で名前の候補が出て、タップで入力されること。設定「ライト／ダーク」で、ステータスバーとナビゲーションバーの色とアイコンの明暗が画面に合うこと（MainActivity の setAppearance と applyColors は PC で動かしていない。コンパイルだけ確認）

既知の制約

- 64 ビットの ARM（arm64）端末だけで動きます。32 ビット端末と x86 端末は対象外です。
- 画面は英語と日本語だけです。日本語は、メニューか設定の「Language / 言語」で選んだときだけ使います（端末の言語には合わせません）。
- QCalc Pro は、課金のある版（Gradle で qcalc.billing=true）でだけ購入できます。build-aab.sh で作る版は課金がなく、Pro の機能をすべて使えます（詳細設計書 3.20）。
- libqalculate のメッセージは英語です。日本語の画面では、よく出る 28 種類（Java とエンジンの文などを含めると 57 種類）だけ画面が日本語にします（詳細設計書 3.14）。英語の画面では、すべて英語で表示します。

- plot()、command()、load()、export() は使えません。グラフはメニューの「グラフ」を使います（2D は変数 x の関数を最大 6 本、3D は変数 x と y の関数を最大 4 つ）。
- 一時的な変数（x = 5 など）と ans2〜ans5 は、アプリを閉じる、またはエンジンが起動し直すと消えます。ans は戻ります。
- 精度（有効桁数）は 200000 桁までです。桁を増やすほど計算に時間がかかり、ζ(3) のような重い関数は確定の制限時間（20 秒）を超えて打ち切られます。入力中の表示は 1.5 秒で打ち切るので、高い精度では「…」になることが多くなります。
- 450,000 バイト（UTF-8）を超える結果は、先頭と末尾を残して途中を省きます（「…」と「長い結果のため、途中を省略しています（全 N 桁）」。版 1.3.0）。クリップボードへのコピーは Android の制限で一度に約 1 MB までなので、実部と虚部が 20 万桁の複素数などはコピーできないことがあります。履歴に全部の桁で残すのは新しい結果から合計 100 万文字まで、アプリを閉じても残る ans は 50 万文字までの値です。
- 為替レートは取得元のサービスに依存します。取得できないときは、前回取得したレート（なければエンジンに組み込まれたレート）で計算します。
- limit() には、libqalculate が「完全ではなく、結果が正しくない場合がある」と必ず警告を付けます（結果は表示します）。dsolve() が解けるのは 1 階の微分方程式だけです。上限が ∞ の総和（無限級数）は計算されず、式のまま返ります。
- 1 文字の名前（a、b、m、t など）は単位として読まれることがあります。解析の式では、変数に x・y・z・n を使います。
- 行列の固有値を直接求める関数はありません。特性方程式 solve(det(M − x × identity(n)) = 0) で求めます（式の例に収録）。行列の階数は rk() です（rank() は順位を返す別の関数）。
- 行列を表の形で表示するのは、2 行以上で要素が 400 個までのものです。それより大きいものと 1 行のベクトルは、エンジンが返す 1 行の文字のまま表示します。
- 端末の文字サイズの設定は反映しません（アプリの設定「文字の大きさ」で 3 段階に変えられます）。
- 試験に使ったスクリプトはソース一式に含まれていません。